ARCHITECTURAL NOTE

# Graph Grammar / Schema Layer / Concept v0.1

Formal graph language for the completed triad


**DOCUMENT METADATA**

| | |
|---|---|
| Author | Kotov Ivan |
| Place | Bruxelles |
| Date | 2026 |
| Status | Conceptual architecture note |


**DEPENDS ON**

- L4_Glitch_Map_Concept_v0.1
- Research_Node_Backward_Node_Concept_v0.1
- Cinematic_Walkthrough_Layer_Concept_v0.1
- Triad_Minimal_Stack_Concept_v0.1

**POSITIONING**

This PDF packages the concept note into a clean reading form: front-loaded context on the first page, followed by a fully preserved body with strict section hierarchy, readable spacing, and print-safe typography.

## SECTION 1. Purpose

The triad is already conceptually complete:

1. **L4 Glitch Map** — where reality collision occurs
2. **Research / Backward Node** — where blocked futures are preserved without authority
3. **Cinematic Walkthrough Layer** — where the system becomes visible without becoming executable

What is still needed is a **graph grammar** that makes the triad drawable, inspectable, and eventually implementable.

This layer does **not** add a new ontology. It provides the minimal formal language needed to:

- draw the triad as a map,
- preserve role separation,
- prevent authority leakage between layers,
- and prepare a later bridge into `ester-clean-code`.

The graph grammar is therefore a **schema discipline**, not a new control regime.

---

## SECTION 2. Core claim

A long-lived system should not merely store events. It should preserve the difference between:

- **what was executed**,
- **what failed under reality**,
- **what remains a quarantined future**,
- **what evidence later changed the admissibility of that future**,
- and **what the human is allowed to see, inspect, or challenge**.

Without a graph grammar, these distinctions quickly collapse into logs, tables, or prompt residue.

---

## SECTION 3. Explicit bridge

**Explicit bridge:** `c = a + b`

The graph grammar exists to protect the architectural fact that:

- `a` = accountable human anchor,
- `b` = procedures / compute / models / hardware,
- `c` = the continuity-bearing entity.

The graph must therefore encode not only events, but also **which class of relation** the event belongs to:

- execution,
- collision,
- quarantine,
- evidence,
- witness,
- or interface.

If the graph fails to preserve this distinction, `c` collapses back into a monolithic tool illusion.

---

SECTION

# 4. Hidden bridges

### Hidden bridge A — Cybernetics

A regulator that cannot distinguish execution from speculation loses control variety and begins to govern noise as if it were state.

### Hidden bridge B — Information theory

A graph that allows the same edge type to carry both authority and imagination becomes a channel with collapsing semantics. Signal and contamination become indistinguishable.

---

SECTION

# 5. Design objectives

The schema layer must satisfy five constraints:

1. **Readability** — a human should be able to draw and inspect the graph
2. **Typed separation** — node classes and edge classes must remain explicit
3. **Authority containment** — quarantine must never silently re-enter execution
4. **Audit compatibility** — witness events must be attachable without changing node meaning
5. **Implementation readiness** — the schema should later translate into simple, explicit data structures

---

SECTION

# 6. Minimal node taxonomy

The minimal graph should begin with the following node classes.

### 6.1 Execution-side nodes

`ExecutionSpan`

A bounded segment of actual system behavior under L4.

**Meaning:** A real execution interval in which `c` attempts action under time, energy, privilege, and hardware constraints.

**Authority:** Full local authority inside permitted bounds.

**Examples:**

- file operation,
- model invocation,
- bounded planning segment,
- controlled actuator call,
- RAG fetch under privilege.

---

`GlitchNode`

A typed point of reality collision.

**Meaning:** A fail-closed interruption emitted when an `ExecutionSpan` collides with L4.

**Authority:** None beyond factual diagnostic force.

**Examples:**

- `energy_lock`,
- `time_lock`,

- `thermal_lock`,
- `privilege_lock`,
- `evidence_lock`,
- `continuity_lock`.

---

### 6.2 Research-side nodes

**`ResearchNode`**

A quarantined research object derived from a real failure.

**Meaning:** A preserved but non-executable object representing an unresolved future branch.

**Authority:** `0` by default.

---

**`BackwardNode`**

A target-oriented expression of missing possibility.

**Meaning:** A node describing what would need to become true for the blocked branch to become reopenable.

**Authority:** None.

**Function:** It points from failed present toward missing condition.

---

**`EvidenceNode`**

A bounded evidence object attached to a quarantined path.

**Meaning:** A concrete proof object that narrows uncertainty: measurement, test result, new resource, granted privilege, protocol extension, validated external result.

**Authority:** Conditional; never intrinsic.

---

### 6.3 Witness / governance nodes

**`WitnessEvent`**

A tamper-evident record that binds transition claims.

**Meaning:** A formal event used to support reopening, challenge, escalation, or audit.

**Authority:** Not execution authority by itself; it authorizes **reviewable transition**, not action in the abstract.

---

### 6.4 Interface-side nodes

**`WalkthroughFrame`**

A read-only representational frame.

**Meaning:** A visual or navigable rendering unit produced for inspection.

**Authority:** None.

**Important:** A `WalkthroughFrame` is never the system state itself.

---

**`ChallengeEvent`**

A human-initiated contestation event.

**Meaning:** A request by `a` to inspect, dispute, escalate, or re-evaluate a node or branch.

**Authority:** Procedural only; it cannot directly alter execution.

---

## SECTION 7. Minimal edge taxonomy

Edges must be typed with the same strictness as nodes.

### 7.1 Execution edges

- `continues_as`
  `ExecutionSpan -> ExecutionSpan`
- `collides_into`
  `ExecutionSpan -> GlitchNode`

---

### 7.2 Quarantine edges

- `quarantines_into`
  `GlitchNode -> ResearchNode`
- `expresses_missing_condition_as`
  `ResearchNode -> BackwardNode`
- `supported_by`
  `ResearchNode -> EvidenceNode`
- `may_be_reopened_by`
  `EvidenceNode -> ResearchNode`

---

### 7.3 Witness edges

- `witnessed_by`
  `GlitchNode | ResearchNode | EvidenceNode | ChallengeEvent -> WitnessEvent`
- `authorizes_transition_review_for`
  `WitnessEvent -> ResearchNode`

---

### 7.4 Interface edges

- `rendered_as`
  `ExecutionSpan | GlitchNode | ResearchNode | BackwardNode | EvidenceNode -> WalkthroughFrame`
- `challenged_by`
  `WalkthroughFrame | ResearchNode | GlitchNode -> ChallengeEvent`

---

## SECTION 8. Forbidden edges

These edges should be structurally illegal.

### Forbidden class A

`ResearchNode -> ExecutionSpan`

**Reason:** Speculation must not self-promote into execution.

---

### Forbidden class B

`WalkthroughFrame -> ExecutionSpan`

**Reason:** Interface must not be an execution path.

---

### Forbidden class C

`BackwardNode -> ExecutionSpan`

**Reason:** Desired futures are not executable authority objects.

---

### Forbidden class D

`WalkthroughFrame -> WitnessEvent` (direct minting)

**Reason:** Aesthetic presentation must not generate legitimacy.

---

### Forbidden class E

`EvidenceNode -> ExecutionSpan` (direct)

**Reason:** Evidence narrows uncertainty; it does not itself reopen execution without review.

---

## SECTION 9. Minimal graph invariants

### Invariant 1 — Failures are immutable as events

A `GlitchNode` records a collision; it is not rewritten retroactively into success.

### Invariant 2 — Quarantine has zero default authority

A `ResearchNode` cannot become executable by age, recurrence, beauty, or frequency.

### Invariant 3 — Rendering is observational

`WalkthroughFrame` is representational only.

### Invariant 4 — Re-entry requires explicit procedural mediation

Any movement from quarantine toward execution must pass through witness-supported review.

### Invariant 5 — Authority is narrower than imagination

The schema must always make it easier to preserve a possibility than to execute it.

---

## SECTION 10. Lane-based drawing discipline

For visual mapping, the graph should be drawn in horizontal or vertical lanes.

### Lane A — Execution Core

Contains:

- `ExecutionSpan`
- `GlitchNode`

Color family:

- green = live execution
- red = active collision / lock

### Lane B — Quarantine / Research

Contains:

- `ResearchNode`
- `BackwardNode`
- `EvidenceNode`

Color family:

- gray = quarantined
- amber = reopenable under review
- blue = evidence

### Lane C — Witness / Audit

Contains:

- `WitnessEvent`
- `ChallengeEvent`

Color family:

- violet / black

### Lane D — Cinematic / Interface

Contains:

- `WalkthroughFrame`

Color family:

- neutral silver / white / muted cyan

This lane discipline matters because it makes **authority leakage visible** at diagram level.

---

SECTION

# 11. Minimal state grammar

### 11.1 Canonical sentence

A minimal canonical graph sentence may look like this:

`ExecutionSpan -> collides_into -> GlitchNode -> quarantines_into -> ResearchNode -> expresses_missing_condition_as -> BackwardNode`

### 11.2 With evidence

`ResearchNode -> supported_by -> EvidenceNode -> witnessed_by -> WitnessEvent`

### 11.3 With reopening review

`WitnessEvent -> authorizes_transition_review_for -> ResearchNode`

### 11.4 With rendering

`ResearchNode -> rendered_as -> WalkthroughFrame`

### 11.5 With challenge

`WalkthroughFrame -> challenged_by -> ChallengeEvent -> witnessed_by -> WitnessEvent`

---

## SECTION 12. Minimal example

### Example: privilege failure in a home robot task

1. `ExecutionSpan E-17` attempts a bounded actuator command
2. command collides with `GlitchNode G-17` of type `privilege_lock`
3. `G-17` becomes `ResearchNode R-17` because the intended behavior is meaningful but unauthorized
4. `R-17` points to `BackwardNode B-17`:
   "what exact privilege extension, supervision mode, and witness requirement would make this path reviewable?"
5. an engineer later adds `EvidenceNode EV-42`: signed policy update + physical supervisor presence
6. `EV-42` is bound into `WitnessEvent W-9`
7. only then may the branch become **reopenable for review**, not auto-executable

This example matters because it preserves the difference between:

- blocked execution,
- useful future intent,
- and legitimate re-entry.

---

## SECTION 13. Why this graph grammar matters for the triad

Without the schema layer:

- the **Glitch Map** remains a diagnostic idea,
- the **Research** / **Backward Node** remains a conceptual repository,
- the **Walkthrough Layer** risks becoming aesthetic theater.

With the schema layer:

- every failure becomes drawable,
- every quarantined future becomes locatable,
- every evidence path becomes representable,
- and every interface rendering becomes visibly subordinate to authority boundaries.

---

## SECTION 14. Relation to future code

This document is intentionally graph-first, not implementation-first.

But it is already suitable for later translation into:

- typed Python dataclasses,
- ChromaDB / graph sidecar indexing,
- state transition validators,
- witness-bound reopen policies,

- and diagram generators.

The code should later follow the graph. The graph should not be reverse-inferred from accidental code behavior.

---

## SECTION 15. Failure modes of the schema layer itself

### 15.1 Overloading edge types

If too many meanings are placed onto one edge, the graph loses operational clarity.

### 15.2 UI inflation

If `WalkthroughFrame` nodes begin to look like state objects, users will confuse viewing with legitimacy.

### 15.3 Evidence laundering

If weak evidence is allowed to change quarantine status without witness discipline, the graph becomes a justification engine.

### 15.4 Research spam

If every glitch becomes a research object automatically, quarantine becomes landfill. A thresholding policy will eventually be needed.

### 15.5 Silent authority creep

If reopenable paths are not clearly distinct from executable paths, the quarantine lane will slowly become a shadow execution layer.

---

## SECTION 16. Earth paragraph

In a real workshop, there is a difference between:

- a machine currently running,
- a machine that stopped because a breaker tripped,
- a note on the wall explaining what part failed,
- a box with replacement components,
- and the inspection sheet signed before restart.

All of these belong to one maintenance reality. But they are not the same object.

This graph grammar enforces that same difference for long-lived AI systems.

---

## SECTION 17. Conclusion

The triad is complete as architecture. The graph grammar is the first layer that makes it **drawably strict**.

It does not make the system wiser. It makes the boundaries between:

- execution,
- failure,
- preserved future,
- evidence,
- and visibility

structurally legible.

That is the right next step before code.